CÓDIGOCLARO · MATERIAL DE ESTUDIO

Desarrollo web: HTML, CSS y JavaScript

Cuaderno DAW · HTML Living Standard · CSS vigente · ECMAScript 2025 · APIs del navegador.

Estructura semántica

Objetivo: Organizar una página de prácticas para que se entienda sin depender de su apariencia.

HTML aporta significado y estructura. Usa títulos jerárquicos, una región principal, navegación con nombre y enlaces cuyo texto explique el destino. El atributo lang ayuda a las tecnologías de asistencia a pronunciar el contenido en español. Un elemento se elige por su función, no por el estilo predeterminado que muestra el navegador.

Código · HTML

```
<!doctype html>
<html lang="es">
<body>
  <header><h1>Prácticas DAM</h1></header>
  <nav aria-label="Principal"><a href="/modulos">Módulos</a></nav>
  <main><article><h2>Bases de datos</h2></article></main>
</body></html>
```

Un documento accesible conserva una jerarquía de encabezados que describe secciones y no el tamaño visual de la letra. header puede identificar contenido introductorio, nav una zona de navegación y main el contenido principal; article sirve para una pieza que tenga sentido por sí misma. El nombre accesible de un enlace debe comunicar su destino incluso fuera del párrafo. Mantén un solo flujo de lectura coherente y comprueba landmarks con lector de pantalla o árbol de accesibilidad. La estructura semántica no fija por sí sola el aspecto visual; CSS lo hace después. Mantén el orden de títulos como una tabla de contenidos y describe el idioma del documento para que el navegador pueda aplicar reglas lingüísticas.

Práctica: Cambia el texto del enlace por “Pulsa aquí”. ¿Qué información pierde una persona que recorre enlaces fuera de contexto?

> Respuesta razonada: Pierde la pista de que el destino es la lista de módulos. Un texto como “Consultar módulos” identifica mejor el enlace sin depender del texto que lo rodea.

Documentación consultada: WHATWG HTML Living Standard

Formularios y validación accesible

Objetivo: Recoger el correo del centro e indicar cómo corregir una entrada incorrecta.

Asocia cada control a una etiqueta visible y coloca las instrucciones cerca del campo. required y type=email ayudan a validar casos comunes en el navegador, pero no son una frontera de seguridad: la petición puede enviarse desde otro cliente. El servidor debe validar formato y regla de negocio de nuevo.

Código · HTML + WAI

```
<form>
  <label for="correo">Correo del centro</label>
  <input id="correo" name="correo" type="email" required>
  <button type="submit">Enviar solicitud</button>
</form>
```

El atributo for de label debe coincidir con el id del control; name determina el dato que el formulario envía. required y type=email aportan validación nativa, pero no impiden que alguien construya otra petición. Cuando falla la validación, explica el problema en texto junto al campo y permite corregirlo sin perder la información introducida. No uses el placeholder como única etiqueta: desaparece al escribir y no reemplaza el nombre del control. La etiqueta visible ayuda a identificar el campo antes y después de escribir. Si se muestra un error, relaciónalo programáticamente con el control y no dependas solo del color; así el mensaje sigue disponible con tecnologías de asistencia.

Práctica: ¿Qué debe hacer el servidor si recibe una petición sin correo aunque la interfaz tenga required?

> Respuesta razonada: Debe rechazarla con un mensaje comprensible. La validación cliente mejora el uso, pero se puede omitir; la etiqueta da nombre al control y el servidor vuelve a comprobar los datos.

Documentación consultada: W3C WAI: formularios · WHATWG HTML Living Standard

Cascada, especificidad y modelo de caja

Objetivo: Corregir una tarjeta cuyo ancho no coincide con el diseño acordado.

La cascada considera origen, importancia, capas, especificidad y orden para elegir una declaración. El ancho se compone de contenido, padding, borde y margen; con border-box, padding y borde se incluyen en el tamaño declarado. Inspecciona estilos calculados antes de añadir reglas más específicas o !important.

Código · CSS Cascade

```
.tarjeta {
  box-sizing: border-box;
  width: 18rem;
  padding: 1rem;
  border: 1px solid #667085;
  color: #243b80;
}
```

La cascada no decide solo por “la regla más específica”: participan origen, importancia, capas, especificidad y orden. DevTools muestra qué declaración ganó y por qué. Con box-sizing:border-box, el ancho declarado incluye contenido, padding y borde; el margen queda fuera. Si una tarjeta mide más de lo esperado, revisa width, padding, borde y reglas heredadas antes de añadir !important. Usa clases con responsabilidades claras para que el arreglo no afecte otras vistas. Las capas de cascada pueden resolver conflictos antes de comparar especificidad. Usa una capa para estilos base y otra para componentes si el proyecto lo necesita; añadir selectores cada vez más largos puede esconder la causa.

Práctica: Dos reglas asignan color a .tarjeta. ¿Qué compruebas antes de aumentar la especificidad?

> Respuesta razonada: Comprueba qué regla gana en el panel de estilos, su capa, importancia y orden. Elimina la regla duplicada si ya no tiene responsabilidad; aumentar especificidad suele crear una cadena difícil de mantener.

Documentación consultada: W3C CSS Cascade Level 5 · W3C · CSS Box Model Level 3

Flexbox, Grid y diseño adaptable

Objetivo: Distribuir tarjetas de módulos sin provocar desbordamiento en móvil.

Flexbox ordena elementos principalmente en una dimensión y Grid define filas y columnas. minmax y auto-fit ajustan columnas al espacio disponible. El diseño debe mantener el orden de lectura del documento y tolerar zoom, textos largos y pantallas estrechas; no añadas un breakpoint para cada anchura intermedia.

Código · CSS Grid

```
.catalogo {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(15rem, 1fr));
  gap: 1rem;
}
.tarjeta { min-width: 0; }
```

Flexbox distribuye elementos en un eje principal y ayuda a alinear una barra; Grid organiza filas y columnas para tarjetas. minmax define un ancho mínimo y un máximo flexible, mientras auto-fit permite adaptar el número de columnas al espacio. min-width:0 puede permitir que contenido largo se contraiga en un grid item. Revisa textos ampliados y zoom además de los anchos de pantalla; un breakpoint no debe alterar el orden semántico de lectura. El tamaño mínimo de una columna afecta a cuándo se produce el desbordamiento. Prueba contenido largo, zoom y una ventana estrecha, no solo una captura de pantalla con frases cortas; el flujo debe seguir legible.

Práctica: Comprueba la cuadrícula a 320 px y 1280 px. ¿Cuántas columnas esperas en cada caso?

> Respuesta razonada: En la ventana estrecha cabe una columna y no debe aparecer desplazamiento horizontal; en la ancha aparecen varias según el espacio. El contenido conserva su orden original.

Documentación consultada: W3C Flexbox Level 1 · W3C Grid Level 2

Valores y comparaciones en JavaScript

Objetivo: Conservar un cero válido cuando se decide si quedan plazas disponibles.

JavaScript distingue valores falsy como 0, cadena vacía, false, null y undefined. || devuelve un operando y puede sustituir un cero válido; ?? solo usa el valor alternativo ante null o undefined. === compara sin coerción implícita. Declara con const si no reasignas la variable y usa let cuando debas cambiar su referencia; const no vuelve inmutable un objeto contenido.

Código · ECMAScript 2025

```
const plazas = 0;
const conOr = plazas || 10;
const conNullish = plazas ?? 10;
console.log(conOr, conNullish); // 10, 0
```

La diferencia entre || y ?? es visible con plazas = 0: el primero elige el valor alternativo porque 0 es falsy, el segundo conserva cero porque solo comprueba null/undefined. Usa === para comparar el mismo tipo de valor sin conversión implícita. const impide reasignar el enlace de la variable, pero un objeto const todavía puede mutarse; evita esa mutación si el componente comparte el mismo objeto con otra parte de la interfaz. Si el dato significa que no existe valor, usa null o undefined según el contrato y valida ambos. Si cero es una cantidad real, conserva cero. No conviertas todos los valores falsy en una misma categoría por comodidad.

Práctica: Si cero significa “completo”, ¿qué operador conserva el dato?

> Respuesta razonada: ?? conserva cero porque no considera falsy todos los valores; || sustituye cero por 10. La regla debe expresar diferencia entre ausencia y valor numérico válido.

Documentación consultada: ECMAScript 2025

Transformaciones de arrays

Objetivo: Filtrar artículos caros y producir sus nombres sin alterar la lista fuente.

filter selecciona elementos y map transforma cada elemento a otro valor; ambos crean resultados nuevos y permiten leer el flujo de datos de izquierda a derecha. Define si el umbral es inclusivo y comprueba array vacío y límite. No compartas una mutación inesperada con otros componentes que usan los mismos datos.

Código · ECMAScript 2025

```
const productos = [
  { nombre: "Teclado", precio: 25 },
  { nombre: "Cable", precio: 8 }
];
const caros = productos.filter(p => p.precio > 20);
const nombres = caros.map(p => p.nombre);
```

filter devuelve una selección y map transforma los elementos de esa selección; ninguno modifica por sí mismo el array original. En el ejemplo, el filtro estricto precio > 20 excluye un artículo que cueste exactamente 20. Define el umbral con el enunciado, y comprueba el array vacío antes de construir mensajes. Si las operaciones se encadenan, nombra resultados intermedios cuando eso facilite explicar qué conjunto entra en cada paso. La comparación >= incluye el umbral y > lo excluye. Esa diferencia debe decidirse con el enunciado y probarse en el propio valor frontera. Mantener productos intacto permite reutilizarlos en otra vista.

Práctica: Cambia > por >= y fija el precio del Teclado en 20. ¿Qué cambia?

> Respuesta razonada: La comparación estricta no selecciona el precio 20; la inclusiva sí. productos mantiene sus dos objetos porque filter construye una selección nueva.

Documentación consultada: ECMAScript 2025

DOM y eventos delegados

Objetivo: Manejar clics sobre tareas que se añaden después de cargar la página.

El DOM es un árbol de nodos y los eventos pueden propagarse hacia los ancestros. Un listener en el contenedor puede identificar el botón activado con closest y delegar el manejo de elementos dinámicos. Inserta texto de usuario con textContent; concatenarlo en innerHTML puede convertir contenido en marcado ejecutable.

Código · DOM Standard

```
const lista = document.querySelector("#tareas");
lista.addEventListener("click", evento => {
  const boton = evento.target.closest("button[data-id]");
  if (!boton || !lista.contains(boton)) return;
  console.log(boton.dataset.id);
});
```

Los eventos de los controles se propagan por el árbol y pueden ser atendidos por un contenedor común. closest busca el botón dentro del objetivo o sus ancestros; contains verifica que pertenece al catálogo. Esa delegación cubre también botones que se agregan después de registrar el listener. Usa textContent para texto no confiable y conserva atributos de datos solo para identificadores que el cliente pueda utilizar; la autorización de una operación sigue siendo responsabilidad del servidor. El evento puede originarse en un hijo dentro del botón, por eso closest es más robusto que comparar el target directamente. Comprueba también que el resultado encontrado pertenezca al contenedor que recibió el evento.

Práctica: Añade una tarea después de registrar el listener. ¿Es necesario enlazarla individualmente?

> Respuesta razonada: No. El clic sube hasta lista y el listener detecta el botón por su atributo. contains evita tratar un botón ajeno como hijo del catálogo.

Documentación consultada: WHATWG · DOM Standard

Fetch, HTTP y validación de respuesta

Objetivo: Separar una respuesta HTTP de error de una respuesta válida con JSON inesperado.

fetch rechaza ante ciertos fallos de red, pero 404 o 500 son respuestas recibidas y requieren comprobar response.ok. Después de analizar JSON valida su forma: un documento correcto puede carecer de los campos esperados. Expón estados de carga y error separados para no presentar datos antiguos como nuevos.

Código · Fetch Standard

```
async function cargar(url) {
  const respuesta = await fetch(url);
  if (!respuesta.ok) throw new Error("HTTP " + respuesta.status);
  const datos = await respuesta.json();
  if (!Array.isArray(datos)) throw new Error("Formato");
  return datos;
}
```

fetch rechaza por ciertos fallos de red, pero una respuesta 404 o 500 sigue siendo una respuesta HTTP y debe comprobarse con ok. La lectura del cuerpo también puede fallar o no cumplir el formato esperado. Separa estado de carga, error y datos válidos para evitar presentar un resultado viejo como si fuera nuevo. Cancela una petición obsoleta si el usuario cambia rápidamente de filtro, y no registres cuerpos que puedan contener información sensible. La validación de Array.isArray solo confirma la forma de la raíz. Para aceptar los datos, revisa además los campos requeridos y sus tipos; un array puede contener objetos incompletos.

Práctica: La respuesta es 200 y el cuerpo es un objeto, no una lista. ¿Qué debe hacer la función?

> Respuesta razonada: Debe rechazar el formato o transformarlo según contrato; no puede tratar el objeto como una lista válida. Comprueba el estado HTTP y el esquema por separado.

Documentación consultada: WHATWG Fetch Standard

Módulos y límites de ejecución

Objetivo: Reutilizar una función de precio tanto en navegador como en Node sin importar el DOM.

Los módulos declaran sus importaciones y exportaciones. La lógica de cálculo puede ser independiente del navegador; la capa de interfaz manipula document y la capa de servidor accede al sistema de archivos. Mantén esas dependencias en sus entornos para que el núcleo sea reutilizable y más fácil de probar.

Código · ECMAScript modules

```
// precios.js
export function conIva(base, tipo = 0.21) {
  return base * (1 + tipo);
}
// app.js
import { conIva } from "./precios.js";
console.log(conIva(100));
```

Los módulos ES exponen dependencias mediante export e import; esto permite leer qué necesita cada fichero y reduce el uso de globals. El navegador resuelve rutas de módulos dentro de su entorno, mientras Node ofrece APIs de servidor como filesystem y no crea document. Mantén el cálculo puro separado de la vista y del acceso a disco: así se puede probar el precio sin abrir una página. Configura el entorno en el que realmente se ejecutará el módulo. Las importaciones declaran dependencias entre módulos y facilitan sustituir el cálculo en una prueba. Evita poner en un módulo común llamadas a APIs disponibles solo en uno de los entornos de ejecución.

Práctica: ¿Por qué precios.js no debe consultar document si también se usa desde Node?

> Respuesta razonada: document pertenece al navegador y no existe en Node. Separar el cálculo de la interfaz deja que ambos entornos compartan la misma regla sin crear globals simulados.

Documentación consultada: WHATWG HTML Living Standard · ECMAScript 2025

Teclado, foco y salida segura

Mini proyecto: catálogo accesible y salida segura

Objetivo: Construir un catálogo responsive de cursos con tarjetas, filtro y un formulario accesible que funcione con teclado.

Código · WAI + DOM

```
const resultado = document.querySelector("#resultado");
function pintar(cursos) {
  resultado.replaceChildren(...cursos.map(curso => {
    const tarjeta = document.createElement("article");
    const titulo = document.createElement("h2");
    titulo.textContent = curso.nombre;
    tarjeta.append(titulo);
    return tarjeta;
  }));
}
```

La revisión final del proyecto comprueba el mismo catálogo en escritorio y móvil, navegación completa con Tab, foco visible, etiquetas enlazadas y texto alternativo útil en imágenes. Prueba que el filtro conserve la lista original y que una lista vacía tenga un mensaje. Usa entradas como `<img>` para confirmar que se muestra texto literal. La validación cliente ayuda a quien usa el formulario, pero el backend aplica las reglas y devuelve errores asociados a campos. El ejemplo usa createElement para construir estructura y textContent para insertar el nombre. Nunca interpoles contenido externo en una cadena HTML; prueba la misma función con nombre vacío, muy largo y con caracteres especiales.

Práctica: Añade un filtro por tecnología y prueba el catálogo con teclado, a 360 px y con un nombre que contenga `<img onerror=...>`. ¿Qué debe verse?

Respuesta razonada: El filtro muestra solo coincidencias y deja un estado claro cuando no hay cursos. El nombre sospechoso aparece literalmente porque textContent no lo interpreta como marcado. Todos los controles tienen etiqueta y foco visible; no aparece scroll horizontal en móvil. Si se envía un formulario, el servidor vuelve a validar los datos y comunica qué campo corregir.

Documentación consultada: WHATWG HTML Living Standard · W3C WAI: formularios · WHATWG · DOM Standard

Fuentes oficiales

WHATWG HTML Living Standard · W3C CSS Cascade Level 5 · W3C Flexbox Level 1 · W3C Grid Level 2 · ECMAScript 2025 · W3C WAI: formularios · WHATWG Fetch Standard · W3C · CSS Box Model Level 3 · WHATWG · DOM Standard